PEtab problems in dMod2

Simon Beyer

2026-09-04

Contents

1 What this vignette covers

PEtab [1] is a file format for parameter estimation problems in systems biology: a YAML manifest, an SBML model [2], and a handful of TSV tables. dMod2 reads such a problem into a fully assembled fit problem in one call, and writes a dMod problem back out in the same format. Both directions are covered here, first as a recipe, then on a worked example, and finally as a table of what the format offers against what dMod2 does with it.

Everything below uses PEtab v2 unless stated otherwise. Version 1 problems are read as well; the importer normalises them onto the same internal shape.

2 What maps onto what

dMod2 composes a fit problem from function objects, `g * x * p` closed by an objective [3]. The importer fills exactly those slots:

PEtab	dMod2
SBML reactions, rules, events	`reactions` (an `eqnlist`), `odemodel`, `x = Xs(odemodel)`
`observables.tsv` formulas	`g = Y(observables)`
`observables.tsv` noise formulas	`e`, the error model, or `NULL` when every noise formula is constant
`parameters.tsv`, `conditions.tsv`	`p = P(trafo)`, one branch per condition
`measurements.tsv`	`dataList`, a `datalist` keyed by condition
`experiments.tsv`	simulation start times and mid-run events
`parameters.tsv` priors	a prior term added to the objective

The composite is `prd = g * x * p` and the objective is `normL2(dataList, prd, errmodel = e)`, with the fixed parameters baked in.

Two conventions are worth stating up front, because they decide what the numbers in `bestfit` mean.

Parameter scales live in the trafo, not in the data. PEtab declares a scale per parameter (`lin`, `log`, `log10`); dMod2 wraps `p` so the optimiser sees the parameter on that scale and the chain rule carries through the composition. `bestfit` is therefore on the PEtab scale, not on the linear one.

Bounds are not a prior. `lowerBound` and `upperBound` reach the fit as `parlower` and `parupper` and never enter the objective. A parameter that declares no `priorDistribution` contributes nothing to the objective value.

3 Importing a problem

```
petab <- importPEtab("problem.yaml", backend = "cppDE")
```

That single call parses the manifest, reads the SBML through libsbml, generates the right-hand side together with its sensitivity equations, compiles them, and returns an object of class `petabproblem`. The arguments worth knowing:

argument	meaning
`yamlPath`	path to the YAML manifest
`backend`	`"cppDE"` (default, handles events and second-order sensitivities) or `"deSolve"`
`compile`	`FALSE` reuses an already compiled model of the same name
`cores`	conditions evaluated in parallel inside one objective call
`modelname`	base name for the generated sources

argument	meaning
`outdir`	where the generated sources and the shared object are written
`optionsOde`, `optionsSens`	solver settings, forwarded to `Xs()`

`outdir` defaults to the working directory, and a problem with many conditions writes one source per condition, so a scratch folder is the better choice. The solver defaults are the backend's; a problem whose reference values were computed at tighter tolerances needs them passed.

The returned object is a plain list; its slots are ordinary dMod objects that can be used, composed and plotted like any others:

slot	what it is
`dataList`	the measurements as a `datalist`
`reactions`	the reaction network as an `eqnlist`
`odemodel`, `x`	compiled model and prediction function
`g`, `e`	observation function and error model
`p`	parameter transformation, one branch per condition
`prd`	the composite `g * x * p`
`obj`	`normL2(dataList, prd, errmodel = e)`, fixed parameters baked in
`bestfit`	`nominalValue` per estimated parameter, on the PEtab scale
`parlower`, `parupper`	bounds, likewise on the PEtab scale

Because the fixed parameters are already inside `obj`, evaluating the likelihood at the published optimum needs nothing else:

```
petab$obj(petab$bestfit)$value
```

and fitting is the usual dMod call, with the bounds handed to the optimiser rather than to the objective:

```
fit <- trust(petab$obj, petab$bestfit, rinit = 1, rmax = 10,
             parlower = petab$parlower, parupper = petab$parupper)
```

3.1 Prerequisites

Reading the SBML model needs `python-libsbml`. dMod2 provisions a managed Python environment through `reticulate` on first use, so no manual setup is required. To use an existing interpreter instead, point `DMOD_LIBSBML_PYTHON` at it. The YAML and TSV readers, `readPetabYaml()` and `readPetabTables()`, work without libsbml.

4 Exporting a problem

There are two entry points, depending on where the problem comes from.

A problem that was imported goes back out with `exportPEtabObject()`. It writes the manifest, the SBML, and every table, and reproduces the tables it read: prior declarations, per-row noise parameters and the period schedule all survive the round trip.

```
exportPEtabObject(petab, dir = "out", formatVersion = "2.0.0")
```

argument	meaning
`petab`	an imported problem, or a list with the same slots

argument	meaning
`dir`	output directory, created if missing
`modelID`	model id in the manifest, defaults to the imported one
`formatVersion`	`"2.0.0"` or `"1"`
`overwrite`	overwrite an existing directory

A problem built by hand goes out with `exportPEtab()`, which takes the dMod objects directly rather than a `petabproblem`:

```
exportPEtab(data = myData, reactions = myEqnlist,
            observables = myObservables,
            p = myTrafo, pouter = myPars,
            errors = myErrorModel,
            lower = myLower, upper = myUpper,
            parameterScale = "log10",
            dir = "out", formatVersion = "2.0.0")
```

The trafo `p` carries the per-condition structure, so the exporter recovers `conditions.tsv` from it. Symbols that the trafo references but that are neither estimated nor declared fixed raise an error rather than being written out silently.

5 A worked example

The package ships the Boehm et al. JAK/STAT5 problem [4], the canonical PEtab benchmark, so the example below runs as written.

```
petab_dir <- system.file("extdata/petab_boehm", package = "dMod2")
list.files(petab_dir)
#> [1] "Boehm.yaml"
#> [2] "experimentalCondition_Boehm_JProteomeRes2014.tsv"
#> [3] "measurementData_Boehm_JProteomeRes2014.tsv"
#> [4] "model_Boehm_JProteomeRes2014.xml"
#> [5] "observables_Boehm_JProteomeRes2014.tsv"
#> [6] "parameters_Boehm_JProteomeRes2014.tsv"
```

The manifest and the tables can be inspected before anything is compiled:

```
yamlPath <- file.path(petab_dir, "Boehm.yaml")
tables   <- readPetabTables(yamlPath)

obs <- tables$observables
data.frame(id = obs$observableId, noise = obs$noiseFormula)
#>            id                        noise
#> 1 pSTAT5A_rel noiseParameter1_pSTAT5A_rel
#> 2 pSTAT5B_rel noiseParameter1_pSTAT5B_rel
#> 3 rSTAT5A_rel noiseParameter1_rSTAT5A_rel
```

The observable formulas are long, so they are shown wrapped rather than as a table column:

```
writeLines(strwrap(paste0(obs$observableId, " = ",
                          obs$observableFormula),
                   width = 68, exdent = 4))
#> pSTAT5A_rel = (100 * pApB + 200 * pApA * specC17) / (pApB + STAT5A
#>     * specC17 + 2 * pApA * specC17)
#> pSTAT5B_rel = -(100 * pApB - 200 * pBpB * (specC17 - 1)) / ((STAT5B
#>     * (specC17 - 1) - pApB) + 2 * pBpB * (specC17 - 1))
#> rSTAT5A_rel = (100 * pApB + 100 * STAT5A * specC17 + 200 * pApA *
```

```
#>     specC17) / (2 * pApB + STAT5A * specC17 + 2 * pApA * specC17 -
#>     STAT5B * (specC17 - 1) - 2 * pBpB * (specC17 - 1))
```

The model has eight STAT5 species, six estimated rate constants, three estimated noise standard deviations and two fixed scaling parameters, all estimated parameters on `log10` scale.

5.1 Import and evaluate

```
wd <- tempfile("petab_vignette_"); dir.create(wd)
petab <- importPEtab(yamlPath, backend = "cppDE",
                     modelname = "boehm_vig", outdir = wd)
```

Compiling the model prints the compiler invocation, which is hidden here. The objective is then evaluated at the published maximum-likelihood estimate:

```
petab$obj(petab$bestfit)$value
#> [1] 276.4441
```

The value is $-2 \log L$ at the published maximum-likelihood estimate.

5.2 Predict and plot

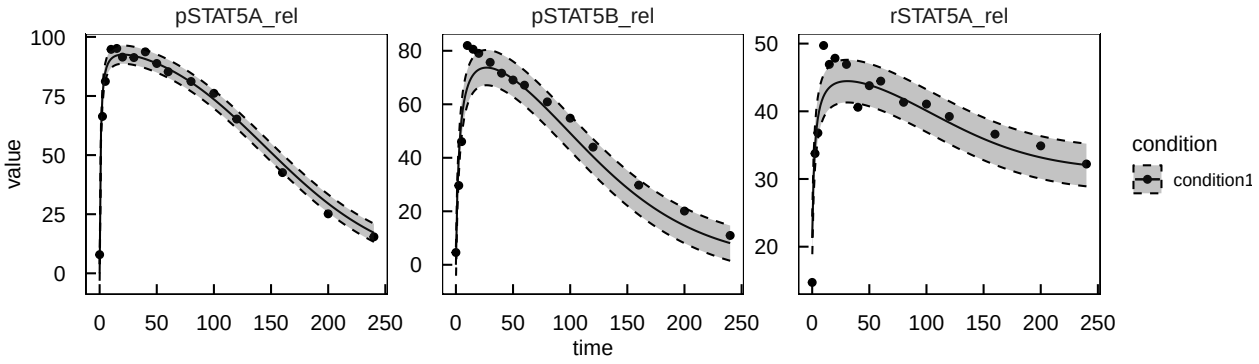
`prd` returns one trace per observable, and `plot()` draws it against the data. The band is one standard deviation, read off the error model rather than reconstructed by hand: `as.data.frame(errfn = )` evaluates it alongside the prediction and returns `sigma` per row.

```
library(ggplot2)

fixed <- attr(petab, "petab_meta")$fixed
times <- seq(0, 240, length.out = 200)
pred  <- petab$prd(times, c(petab$bestfit, fixed), deriv = FALSE)

prdout <- as.data.frame(pred, errfn = petab$e)

plot(pred, petab$dataList) +
  geom_ribbon(data = prdout,
              aes(x = time, ymin = value - sigma, ymax = value + sigma),
              linetype = "dashed", alpha = 0.3)
```

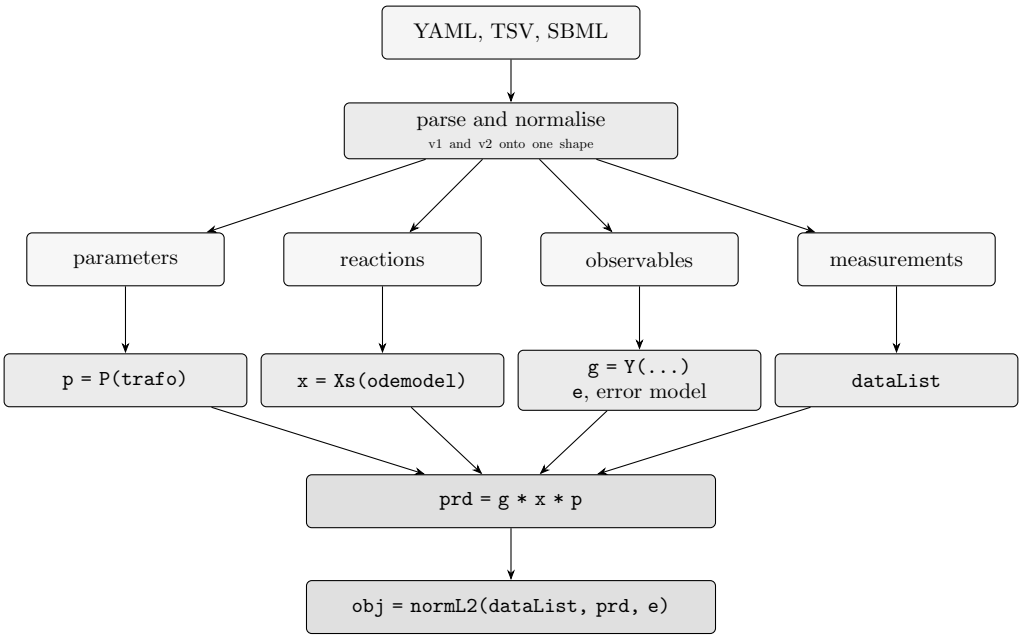


5.3 Write it back out

The imported problem exports as a v2 problem in one call. Boehm arrives as v1, so its `parameterScale` is linearised on the way out, which the exporter warns about.

```r
exportDir <- file.path(wd, "export"); dir.create(exportDir)
exported <- exportPEtabObject(petab, dir = exportDir,
                              formatVersion = "2.0.0",
                              overwrite = TRUE)
list.files(exportDir)
#> [1] "boehm_vig.xml"               "boehm_vig.yaml"
#> [3] "conditions_boehm_vig.tsv"    "experiments_boehm_vig.tsv"
#> [5] "measurements_boehm_vig.tsv" "observables_boehm_vig.tsv"
#> [7] "parameters_boehm_vig.tsv"
```

6 How the importer works



Six pieces are not obvious from that picture.

Extensive against intensive flux. SBML kinetic laws carry units of amount per time, dMod2 stores rates as concentration per time. Each kinetic law is divided by the volume of its home compartment on import, $\text{rate}_{\text{dMod}} = K_{\text{SBML}}/V_{\text{home}}$, and multiplied by it again on export. The home compartment is the shared compartment of the educts, falling back to the products for a pure synthesis. Assignment rules are inlined into rates and initial expressions before the `eqnlist` is built.

Experiments are period sequences. An experiment is an ordered list of `(time, conditionId)` pairs. A leading `-inf` period is the preequilibration and becomes a `Pequil` stage. The first finite period sets the simulation start, which reaches the objective as `normL2(t0 = )`, so initial values take effect there rather than at zero. Every later period becomes an event that applies its condition overrides at that time. There is no limit on the number of periods.

Event targets that are not states. Solvers assign only to states, but PEtab and SBML both allow an event to assign to a parameter or to a compartment volume. Such a target is promoted to a state with rate zero, keeping its SBML default as the initial value; the export takes the promotion back.

Placeholders are per observable. `measurements.tsv` may override `observableParameter${k}_${id}` and `noiseParameter${k}_${id}` per row. Placeholders carry the observable id as a suffix, so substitutions never collide across observables. Rows inside one condition merge into a single sub-condition whose trafo carries one substitution per observable; a new sub-condition appears only when the same observable turns up

with conflicting values. Numeric noise folds into the data column and leaves the error model `NULL`, which is `normL2`'s fast path; symbolic noise becomes an inner parameter of the trafo.

Steady-state measurements. A measurement at `time = inf` is evaluated on the equilibration horizon that `Pequil` integrates to, and is written back as `inf` on export.

7 Priors, and telling them from the likelihood

A parameter row may declare a `priorDistribution` with its `priorParameters`. dMod turns each one into a term on the objective's $-2\log$ scale and adds it to the likelihood, so `obj()` returns $-2\log$ posterior, not $-2\log$ likelihood.

The two parts stay separable. The prior contribution is reported in an attribute of the result, so subtracting it recovers exactly the quantity PEtab publishes as `llh`:

```
res <- petab$obj(petab$bestfit)
prior <- attr(res, "prior")

c(total      = unname(res$value),
  prior      = if (is.null(prior)) 0 else unname(prior),
  likelihood = unname(res$value) - if (is.null(prior)) 0 else unname(prior))
#>      total      prior likelihood
#>   276.4441     0.0000   276.4441
```

Boehm declares no prior, so the attribute is absent and the two agree. For a problem that does declare one, `total` is $-2\log$ posterior and `likelihood` is what a published `llh` can be compared against.

Bounds are not a prior. `lowerBound` and `upperBound` reach the fit as `parlower` and `parupper` and never enter the objective. A parameter with bounds but no declared distribution contributes nothing. This differs from implementations that read bounds as an implicit uniform prior; there the reported posterior carries a constant that dMod leaves out.

Truncation. PEtab truncates every prior to the parameter bounds and renormalises it to the mass between them. dMod does the same. The renormalisation is a constant in the parameter, so it shifts the objective value without touching gradient or Hessian, and the fit is unaffected.

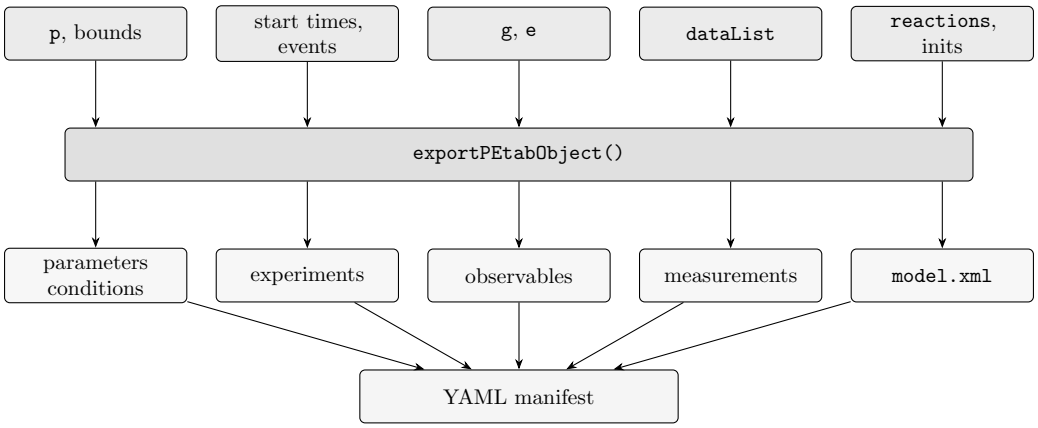
The families, and where their parameterisation bites.

`priorDistribution`	parameters	note
`uniform`	a, b	density $1/(b-a)$ on $[a, b]$
`normal`	mean, sd	
`laplace`	mu, b	
`cauchy`	x0, gamma	
`chisquare`	df	
`exponential`	**scale**	not a rate
`gamma`	shape, **scale**	not a rate
`rayleigh`	sigma	
`log-normal`	meanlog, sdlog	density in the parameter, so it carries the change of variables
`log-laplace`	mu, b	likewise
`log-uniform`	a, b	likewise

Each spelling also exists with a `parameterScale` prefix. Those act on the parameter as the optimiser sees it, i.e. after the scale wrap, and the plain spellings act on the linear value. A plain `normal` or `laplace` on a non-`lin` `parameterScale` is rejected rather than guessed at: the two readings differ, and picking one silently would move the optimum.

8 How the exporter works

The reverse direction reads the same slots and writes one table each. What is not a plain read-back is the model: the kinetic laws are multiplied by their home compartment volume again, promoted event targets go back to being parameters or compartments, and the period schedule is reconstructed from the simulation start times and the mid-run events.



A problem built by hand goes through `exportPEtab()` instead, which takes the same pieces as separate arguments rather than reading them off a `petabproblem`.

9 Feature coverage

Measured against the official PEtab test suite: all 31 v2 cases reproduce the published likelihood, and all 31 survive an export-to-v2 and reimport round trip.

Beyond the test suite, the Benchmark-Models collection was run end to end: 33 of its 35 problems import, evaluate their objective with derivatives, export to v2 and reimport. `Froehlich_CellSystems2018` is left out because each of its 9570 conditions produces its own generated transformation, and `Laske_PLOSComputBiol2019` because one of its observables divides by a sum of species that is zero at the initial time.

9.1 SBML model

feature	dMod2	notes
reactions and kinetic laws	yes	divided by the home compartment volume
compartments, multiple compartments	yes	first class on `eqnlist`
species in amounts or concentrations	yes	`hasOnlySubstanceUnits` respected
`<initialAssignment>`	yes	on species, parameters and compartments
`<assignmentRule>`	yes	inlined before the `eqnlist` is built
`<rateRule>`	yes	on species, and on parameters, which become states
`<event>`, time trigger	yes	rewritten as a dMod event
`<event>`, state trigger	yes	as a root condition
event assigning to a compartment	yes	contained species rescale, amounts conserved
`<functionDefinition>`	yes	inlined by libsbml
`piecewise` in a formula	yes	grouped into a symbolic piecewise

feature	dMod2	notes
`<algebraicRule>`	no	dMod2 has no DAE solver
model languages other than SBML	no	only SBML is read

9.2 PEtab tables

feature	dMod2	notes
`parameterScale`	yes	`lin`, `log`, `log10`, folded into the trafo
`observableTransformation`	yes	`lin`, `log`, `log10` per observable
`conditions.tsv`, long format	yes	pivoted to wide for the trafo builder
condition target on a parameter or compartment	yes	promoted to a state when an event assigns to it
per-row `observableParameters`, `noiseParameters`	yes	numeric and symbolic
noise formula naming its own observable	yes	resolved from the observable equations
`experiments.tsv`, any number of periods	yes	preequilibration, start time, mid-run events
measurement at `time = inf`	yes	evaluated at the equilibration horizon
`mapping.tsv`	yes	textual rewrite of entity ids
several models in one manifest	yes	composed with `+`, condition keys prefixed
priors, all eleven families	yes	truncated to the bounds and renormalised
model without species	yes	a parameter-only problem
`visualisation.tsv`	no	ignored

9.3 Noise distributions

dMod2's objective is `normL2`, so the noise model maps as follows.

`noiseDistribution`	dMod2	notes
`normal`	yes	direct
`log-normal`	yes	through `observableTransformation = log`
`laplace`, `log-laplace`	no	rejected at import, an L1 residual is not implemented

9.4 Export caveats

caveat	notes
`parameterScale` on a v2 export	linearised on disk with one warning, v2 has no such column
`log10` observable transformation on a v2 export	rewritten as `log-normal` noise, v2 has no `log10`
`<algebraicRule>` in the source model	absent from the dMod model and therefore from any SBML written back

caveat	notes
experiment identity	experiments are keyed by their starting condition, so the exported `experimentId` need not match the imported one

References

[1] L. Schmiester *et al.*, "PEtab: Interoperable specification of parameter estimation problems in systems biology," *PLOS Computational Biology*, vol. 17, no. 1, p. e1008646, 2021, doi: 10.1371/journal.pcbi.1008646.

[2] M. Hucka *et al.*, "The systems biology markup language (SBML): A medium for representation and exchange of biochemical network models," *Bioinformatics*, vol. 19, no. 4, pp. 524–531, 2003, doi: 10.1093/bioinformatics/btg015.

[3] D. Kaschek, W. Mader, M. Fehling-Kaschek, M. Rosenblatt, and J. Timmer, "dMod: Efficient parameter estimation and uncertainty analysis for dynamic models," *Journal of Statistical Software*, vol. 88, no. 10, pp. 1–32, 2019, doi: 10.18637/jss.v088.i10.

[4] M. E. Boehm, L. Adlung, M. Schilling, S. Roth, U. Klingmüller, and W. D. Lehmann, "Identification of isoform-specific dynamics in phosphorylation-dependent STAT5 dimerization by quantitative mass spectrometry and mathematical modeling," *Journal of Proteome Research*, vol. 13, no. 12, pp. 5685–5694, 2014, doi: 10.1021/pr5006923.